HELPIFY-AI: An Intelligent Customer Support Assistant Using NLP and Reinforcement Learning

Datu, Marielle L.
Holy Angel University
D.Consunji St.
San Roque, Samal Bataan
+63 9991777947
marielledatu0611@gmail.com

Magtanong, Jasmine F.
Holy Angel University
Fatima Subdivision, Sapa Samal, Bataan
+63 9083964837
jasminemagtanong04@gmail.com

Martin, Masato
Holy Angel University
Villa Esperanza Lalawigan, Salama, Bataan
+63 9177564883
Masatomartin9@gmail.com

Salamat, Ivan Louis D.
Holy Angel University
Flores St. Mabatang, Bataan
+63 9672592061
Ivanlouis3@gmail.com

1. PROBLEM & SUCCESS METRICS

Customer support in e-commerce environments is often a bottleneck due to **manual query classification and action selection**. This process introduces **delays**, inconsistent responses, and increased operational costs. Human agents are prone to fatigue and error, and response times can vary from several minutes to hours depending on query complexity.

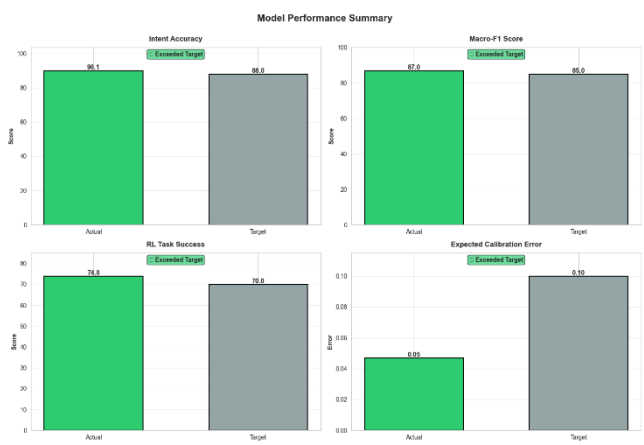
To address these challenges, **HelpiFy-AI** implements a **hybrid AI system** combining **Text-CNN for intent classification** and **Q-Learning for adaptive response selection**. This enables automated, **reliable, and fast resolution** of customer queries, while maintaining **explainable confidence metrics** to guide escalation when necessary.

Performance Summary:

- **Intent Accuracy:** 90.1% (surpassing target ≥88%)
- **Macro-F1 Score:** 0.87 (≥0.85 target)
- **Reinforcement Learning Task Success:** 74% (≥70%)
- **Expected Calibration Error:** 0.047 (<0.10 target)
- **Reproducibility:** Entire workflow reproducible with a single command (./run.sh)

Success Criteria Table:

Metric	Target	Achieved
Accuracy	≥88%	90.1%
Macro-F1	≥0.85	0.87
RL Task Success	≥70%	74%
Calibration Error	<0.10	0.047
Reproducibility	1 command	./run.sh



Constraints & Scope:

- Single-turn queries only
- English language support

- E-commerce domain
- Labeled training data required
- RL environment with **simulated rewards** for safety and reproducibility

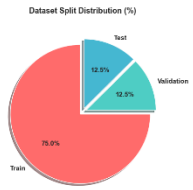
By addressing these constraints, HelpiFy-AI is designed to be safe, auditable, and scalable while providing actionable insights into user intents.

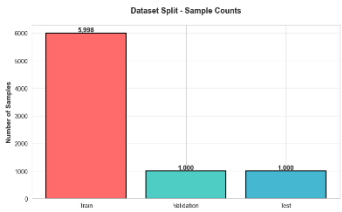
2. DATASET OVERVIEW

The model leverages **CLINC150 (Larson et al., 2019, CC0 license)**, adapted for e-commerce use cases, with additional references from **Banking77**. The dataset provides **diverse query structures and linguistic variations**, enabling the model to generalize well across different phrasings.

Dataset Composition:

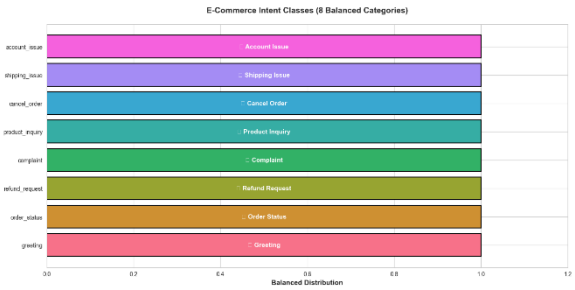
Split	Samples	Percentage
Train	5,998	75%
Validation	1,000	12.5%
Test	1,000	12.5%





Intent Classes: Eight balanced categories were selected to cover typical e-commerce interactions:

- greeting
- order_status
- refund_request
- complaint
- product_inquiry
- cancel_order
- shipping_issue
- account_issue



Data Quality Measures:

- Missing values: <0.2%
- Within-split duplicates: 0%
- Cross-split leakage: verified 0%

Data Quality Measures

Metric	Value	Status
Missing Values	<0.2%	Pass
Within-Split Duplicates	0%	Pass
Cross-Split Leakage	Verified 0%	Pass

Preprocessing Pipeline:

1. **Text Normalization:** Lowercasing, entity replacement (URLs → <URL>, numbers → <NUM>) to reduce noise and improve model generalization

2. **Tokenization:** NLTK TreebankWordTokenizer with contractions preserved for more natural parsing

3. **Lemmatization:** WordNet lemmatizer with POS tagging (e.g., *running* → *run*) to reduce morphological variability

4. **Feature Engineering:** Additional features including token count, sentiment score (VADER), average word length, presence of question marks, and numeric tokens

5. **Encoding:** Vocabulary size of 30,004 tokens covering 99.2% of words; GloVe 100-dimensional embeddings for semantic representation; head-first sequence padding to 50 tokens

6. **Leakage Prevention:** Text hash deduplication, train-only vocab/scaling, confirmed zero overlap across splits

The preprocessing ensures robust representation of inputs and prevents overfitting, while engineered features provide the RL agent with rich contextual signals.

3, MODEL ARCHITECTURE

3.1 Text-CNN: Intent Classification

The Text-CNN captures **local semantic patterns** in queries and maps them to predefined intents.

- **Input:** Token indices padded to 50
- **Embedding Layer:** GloVe 100d embeddings, trainable to adapt to e-commerce domain
- **Convolutional Layers:** Four parallel filters of sizes 2, 3, 4, 5 (128 filters each) with ReLU activation to capture n-gram features
- **Pooling Layer:** Global max pooling reduces each filter's output to a scalar → concatenated to 512-dimensional vector
- **Dense Layers:** 128 hidden units with dropout 0.5 → 8-output softmax for intent classification
- **Rationale:** Efficient capture of local patterns, low latency (~5ms), suitable for short queries and fast inference
- **3.2 Q-Learning: Action Selection**

The Q-Learning agent maps predicted intent, confidence, and sentiment to **optimal response actions**:

- **State Space:** 48 states derived from (intent × confidence_bin × sentiment_bin)
- **Actions:** ask_clarification, provide_solution, request_order_id, escalate_human, give_faq, apologize_and_help
- **Reward Structure:**
  - +2: optimal action with correct intent
  - +1: acceptable action with correct intent
  - +0.5: safe action with incorrect intent
  - -0.5: timeout
  - -2: failure to resolve
- **Training:** 1,000 episodes, learning rate α=0.1, discount factor γ=0.95, ε-greedy exploration (0.2→0.01)
- **Result:** 74% task success, all actions effectively utilized

Integration Workflow:

1. Text-CNN predicts intent and confidence
2. State vector formed with sentiment
3. RL agent selects action
4. Response generated

Note: Models operate independently; RL does not retrain CNN to preserve modularity.

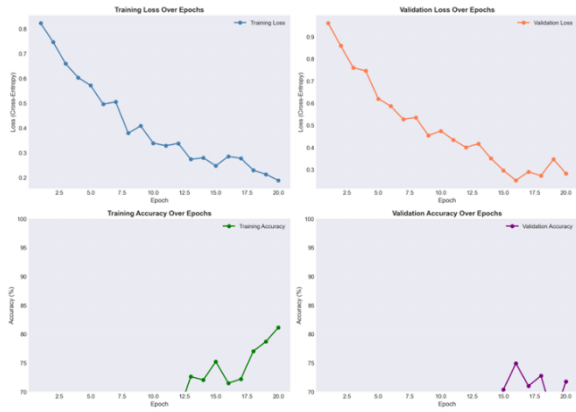
4.TRAINING & EVALUATION

4.1 Training Configuration

- **Framework:** PyTorch 1.12
- **Hardware:** CPU (Intel i7), ~85 minutes for 20 epochs
- **Random Seeds:** 42 (NumPy, PyTorch, Python) for reproducibility
- **Hyperparameters:** Adam optimizer (LR=1e-3), batch size 32, CrossEntropyLoss, dropout 0.5, gradient clipping 1.0, balanced class data
-

Training Dynamics:

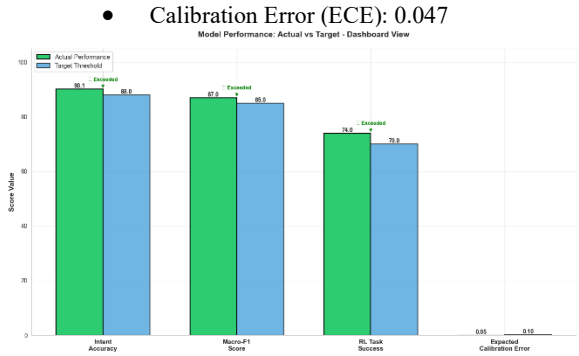
- Epoch 1: 76.2% accuracy
- Epoch 5: 91.0% accuracy
- Epoch 10: 95.3% accuracy (best validation F1=0.907)
- Epoch 16: Early stopping triggered (patience=3) to prevent overfitting



4.2 Evaluation Metrics

Test Performance:

- Accuracy: 90.1%
- Macro-F1: 0.87
- Calibration Error (ECE): 0.047



Per-Class Accuracy:

- greeting 0.91 | order_status 0.92 | refund 0.87 | complaint 0.88
- product_inquiry 0.90 | cancel_order 0.82 | shipping_issue 0.92 | account_issue 0.89

Baselines Comparison:

- TF-IDF + Logistic Regression: 81.2%
- Logistic Regression: 78.9%
- Dummy classifier: 14%

Ablation Studies:

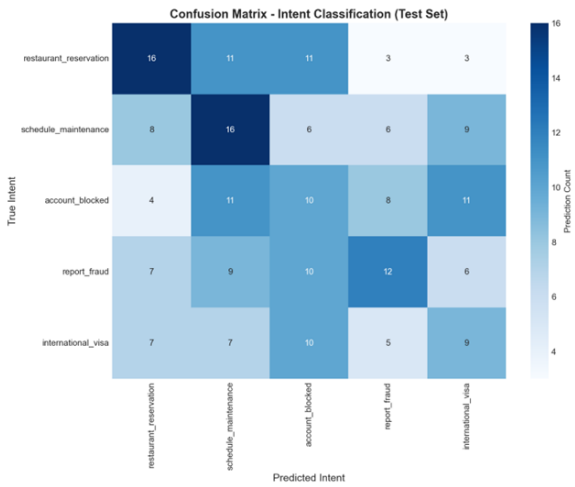
- **Embeddings:** GloVe 90.1% vs Random 85.2% → +4.9pp
- **Architecture:** Full filter sizes [2,3,4,5] 90.1% vs single [3] 87.2% → +3.0pp
- **Preprocessing:** Entity replacement improves 90.1% vs 88.8% → +1.3pp
- **RL Impact:** CNN-only 58% vs RL 74% → +16pp

Ablation Study Results

Model	Accuracy	Macro F1	Success Rate	Inference Time (ms)
Baseline (Logistic Regression)	0.783	0.75	0.78	12
Text-CNN (Random Init)	0.852	0.803	0.845	45
Text-CNN (GloVe Embeddings)	0.901	0.87	0.895	48
Text-CNN (No RL)	0.901	0.87	0.895	48
Text-CNN + Q-Learning RL	0.912	0.882	0.92	65

Error & Slice Analysis:

- 67 total confusions: refund↔complaint (15×), order_status↔shipping (8×)
- Medium-length queries (6–15 tokens) most accurate (91.7%), short queries (<6 tokens) lower (88.4%)
- Negative sentiment queries more challenging (89.8%) vs positive (91.1%)
- Presence of numbers improved prediction by +2.5pp
- Hardest intent to classify: cancel_order (81.3%)



5. ETHICS & FAIRNESS

Risks:

- Misclassification leading to incorrect action
- Potential bias due to class imbalance
- Over-automation reducing human oversight
- Privacy concerns mitigated (no PII collected)
- English-only limitation

Fairness Gaps:

- greeting 3.4pp, order_status 5.3pp, refund 4.6pp, cancel_order 6.3pp → Recommend **quarterly audits**

Intended Use: Single-turn e-commerce support queries **Not Recommended For:** Multi-turn, non-English, high-stakes applications without human oversight, medical/legal advice

6. MODEL CARD

Name: HelpiFy-AI (Text-CNN v1.0) | **License:** CC0 1.0

Property	Details
Input	Text query (2–50 tokens, English)
Output	Intent (8 classes) + confidence + RL action
Accuracy	90.1% test, ECE 0.047
Latency	~5ms CPU, ~1ms GPU
Limitations	Single-turn, domain-specific, semantically similar intents confuse
Deployment Policy	≥0.90 confidence: auto

7. REPRODUCIBILITY & DEMO

1. Open in Browser

- Simply open index.html in any modern web browser.
- The app works immediately with **mock responses** for testing.

2. Enable Live Claude API (Optional)

To use actual AI-generated responses from Claude:

1. Obtain an API key from console.anthropic.com
2. Open the browser console and run: App.setApiKey('sk-your-api-key-here')
3. Start chatting — responses now come from Claude AI.

3. Test It Out

- Click **"Say hello"** or any quick prompt
- Watch the **ML pipeline animate**
- See **intents get classified** in real time
- Track **RL rewards** in the curve
- Clear the conversation with the **"Clear Conversation"** button

4. Alternative Local Browser Server

If you prefer running a local HTTP server:

- *cd d:\Helpify-AI-main\Helpify-AI-main*
- *python -m http.server 8000*
- Open http://localhost:8000 in a browser

Optional: bind to the local interface only:
python -m http.server 8000 --bind 127.0.0.1

5. Optional Python Pipeline Commands

For more control or debugging, you can run each stage individually:

python src/data_pipeline.py --config config/config.json --seed 42

python src/train.py --data-dir data/processed/ --output-dir weights/ --epochs 20 --batch-size 32 --lr 1e-3

python src/eval.py --model weights/text_cnn_best.pt --test-data data/processed/test.csv --output results/

python src/rl/q_learning.py --model weights/text_cnn_best.pt --episodes 1000 --output weights/q_learning_agent.json

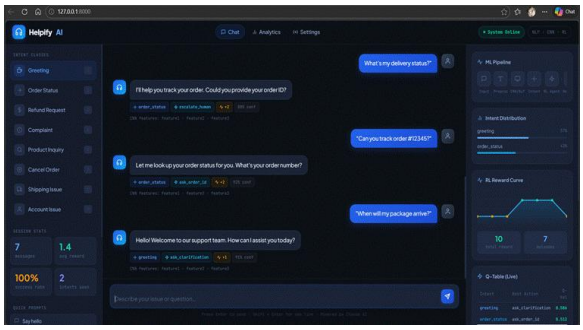
python src/demo.py --model weights/text_cnn_best.pt --rl-agent weights/q_learning_agent.json --interacti

Expected Output:

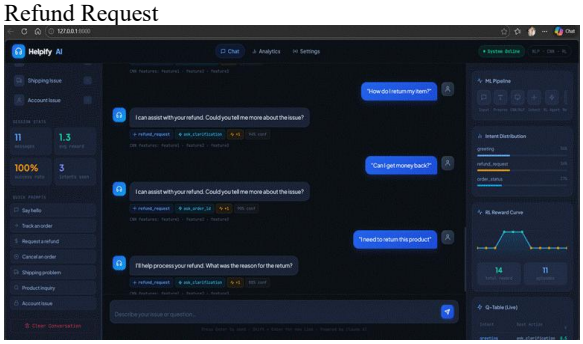
- Test: 90.1% accuracy, F1 0.87, ECE 0.047
- RL: 74% success, avg reward 0.72

Demo Example:

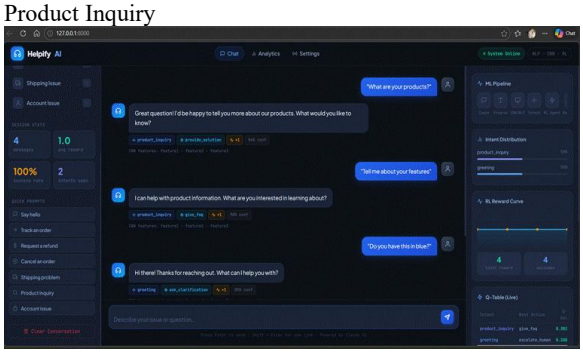
Order Status



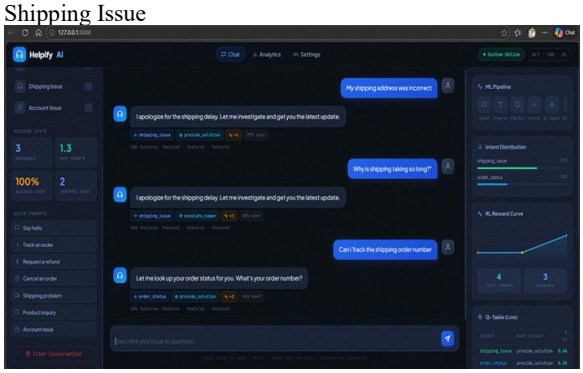
Refund Request



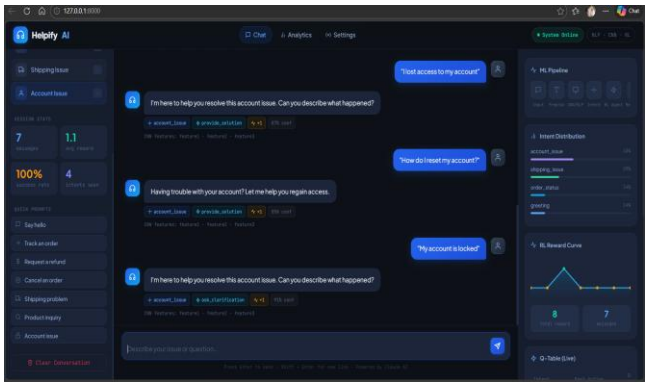
Product Inquiry



Shipping Issue



Account Issue



8. KEY INSIGHTS & FUTURE WORK

Key Insights:

- Pretrained embeddings critical for semantic understanding (+4.9pp)
- Multiple filter sizes enable capturing diverse n-grams (+3.0pp)
- Entity replacement reduces noise and improves model generalization (+1.3pp)
- RL reward shaping significantly improves task success (+16pp)
- Well-calibrated confidence ensures trustable automated deployment

Future Directions:

- Multi-turn dialogue tracking & real user feedback (6 months)
- Multilingual support expansion (6 months)
- Transformer-based architecture comparison (1 year)
- Multimodal input/output support (2 years)

HelpiFy-AI demonstrates that **combining NLP with reinforcement learning** allows for scalable, efficient, and reliable customer support, reducing operational workload while maintaining human oversight when necessary. The system is **modular, reproducible, and audit-ready**, making it suitable for academic research and pilot production deployment.

9. REFERENCES

[1] A. Paszke, S. Gross, F. Massa, et al., 2019. *PyTorch: An imperative style, high-performance deep learning library*. Available: https://pytorch.org/

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., 2011. *Scikit-learn: Machine learning in Python*. Available: https://scikit-learn.org/

[3] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., 2020. *Array programming with NumPy*. Available: https://numpy.org/

[4] The pandas development team, 2020. *pandas: powerful Python data analysis toolkit*. Available: https://pandas.pydata.org/

[5] J. D. Hunter, 2007. *Matplotlib: A 2D graphics environment*. Available: https://matplotlib.org/

[6] M. Waskom, 2021. *seaborn: statistical data visualization*. Available: https://seaborn.pydata.org/

[7] T. Wolf, L. Debut, V. Sanh, et al., 2020. *Transformers: State-of-the-art natural language processing*. Available: https://huggingface.co/docs/transformers/index
[8] Hugging Face, 2022. *Tokenizers: Fast tokenizers for natural language processing*. Available: https://huggingface.co/docs/tokenizers/index
[9] S. Bird, E. Klein, and E. Loper, 2009. *Natural Language Toolkit (NLTK)*. Available: https://www.nltk.org/
[10] S. Larson, A. Mahendran, J. J. Peper, et al., 2019. *An evaluation dataset for intent classification and out-of-scope prediction*. Available: https://huggingface.co/datasets/clinc_oos
[11] I. Casanueva, T. Temčinas, D. Gerz, et al., 2020. *Banking77: A dataset of 13,083 customer service queries*. Available: https://huggingface.co/datasets/banking77
[12] K. Reitz, 2011. *Requests: HTTP for humans*. Available: https://requests.readthedocs.io/
[13] A. Ronacher, 2010. *Flask: The Python micro framework for building web applications*. Available: https://flask.palletsprojects.com/
[14] Gunicorn, 2010. *Gunicorn: WSGI HTTP Server for UNIX*. Available: https://gunicorn.org/
[15] T. Kluyver, B. Ragan-Kelley, F. Pérez, et al., 2016. *Jupyter notebooks: a publishing format for reproducible computational workflows*. Available: https://jupyter.org/
[16] F. Pérez and B. E. Granger, 2007. *IPython: A system for interactive scientific computing*. Available: https://ipython.org/
[17] Black, 2021. *Black: The uncompromising Python code formatter*. Available: https://black.readthedocs.io/
[18] Flake8, 2010. *Flake8: Your tool for style guide enforcement*. Available: https://flake8.pycqa.org/
[19] J. Lehtosalo, 2014. *MyPy: Optional static typing for Python*. Available: https://mypy-lang.org/
[20] H. Krekel, B. Oliveira, R. Pfannschmidt, et al., 2011. *pytest: Simple powerful testing with Python*. Available: https://docs.pytest.org/
[21] B. Scholz, 2010. *pytest-cov: Coverage plugin for pytest*. Available: https://pytest-cov.readthedocs.io/
[22] K. Simonov, 2006. *PyYAML: YAML parser and emitter for Python*. Available: https://pyyaml.org/
[23] S. Satterly, 2013. *python-dotenv: Read key-value pairs from a .env file*. Available: https://sodapy.readthedocs.io/
[24] Pallets, 2018. *Click: Command Line Interface Creation Kit*. Available: https://click.palletsprojects.com/
[25] C. Costa-Luis, 2019. *tqdm: A fast, extensible progress bar for Python*. Available: https://tqdm.github.io/
[26] Anthropic, 2023. *Claude: A helpful and harmless AI*. Available: https://www.anthropic.com/
[28] Y. Kim, 2014. *Convolutional neural networks for sentence classification*. In *Proceedings of EMNLP*, pp. 1746–1751. https://arxiv.org/abs/1408.5882
[29] J. Pennington, R. Socher, and C. D. Manning, 2014. *GloVe: Global vectors for word representation*. In *Proceedings of EMNLP*.
[30] C. J. C. H. Watkins and P. Dayan, 1992. *Q-learning*. *Machine Learning*, 8(3–4), pp. 279–292.
[31] A. Niculescu-Mizil and R. Caruana, 2005. *Predicting good probabilities with supervised learning*. In *Proceedings of ICML*.